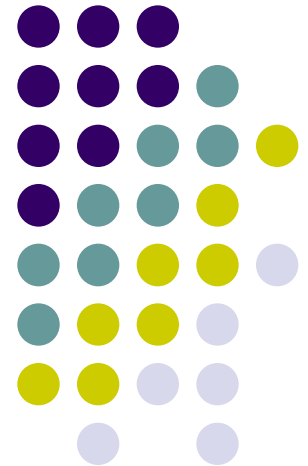
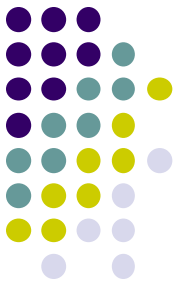


Basic Structure of Computers

Module I

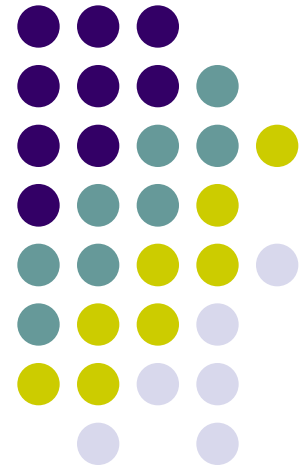




Overview

- Functional Units
- Bus Structures
- Performance
- Overflow In Integer Arithmetic: Numbers, Arithmetic Operations And Characters,
- Memory Locations And Addresses,
- Memory Operations, Instructions And Instruction Sequencing,
- Addressing Modes,
- Encoding Of Machine Instructions.

Functional Units



Functional Units

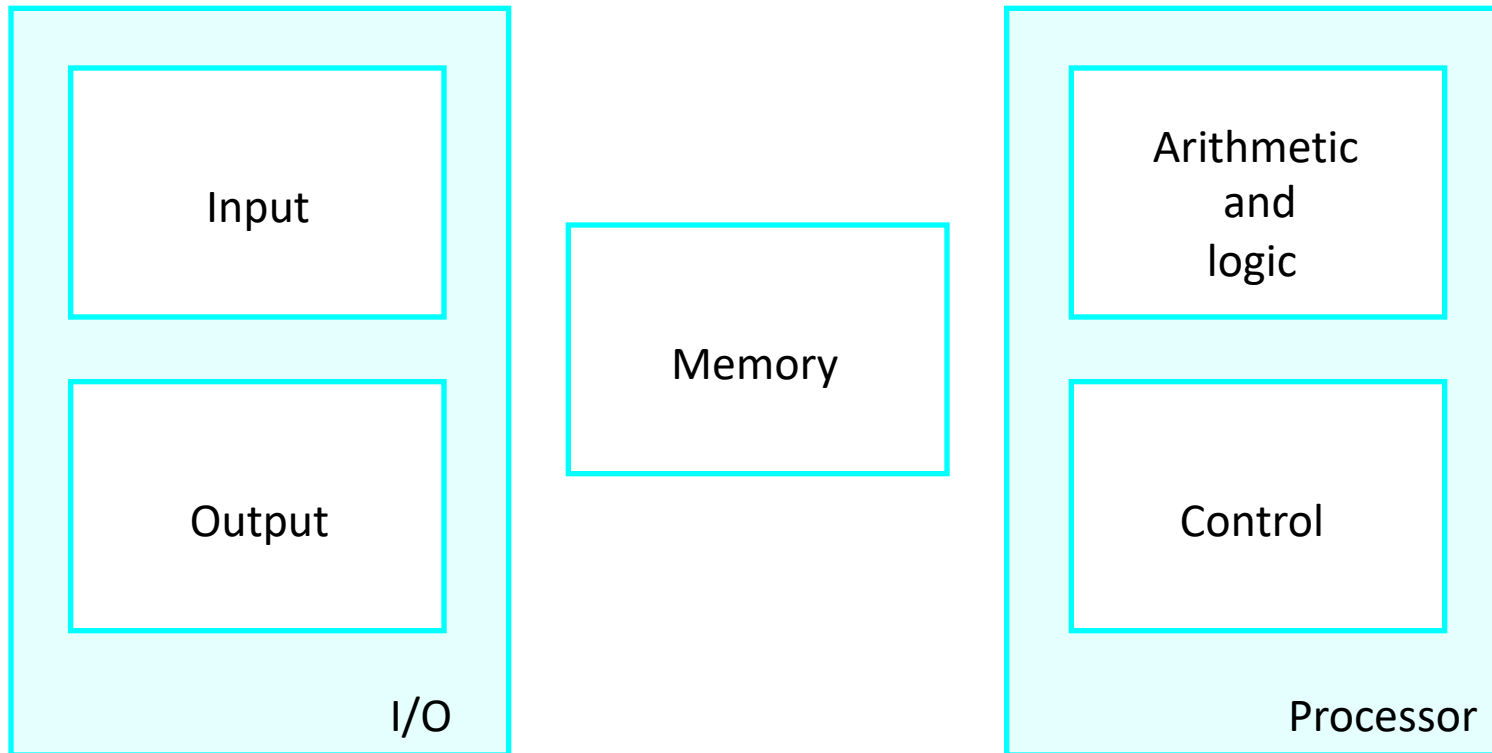
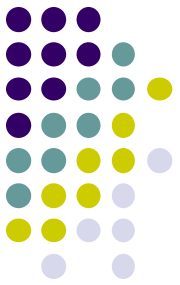
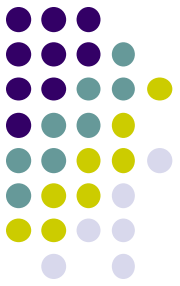


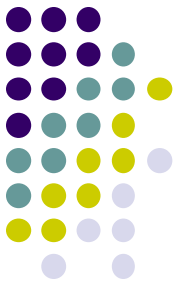
Figure 1.1. Basic functional units of a computer.

Akshata S. Bhayyar, Asst. Prof, CSE Dept. RIT

Information Handled by a Computer



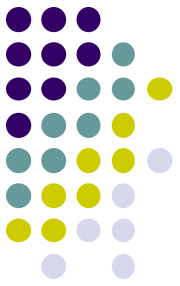
- Instructions/machine instructions
 - Govern the transfer of information within a computer as well as between the computer and its I/O devices
 - Specify the arithmetic and logic operations to be performed
 - Program
- Data
 - Used as operands by the instructions
 - Source program
- Encoded in binary code – 0 and 1



Memory Unit

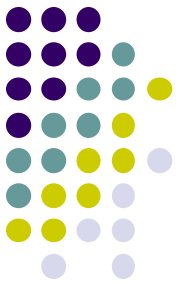
- Store programs and data
- Two classes of storage
 - Primary storage
 - ❖ Fast
 - ❖ Programs must be stored in memory while they are being executed
 - ❖ Large number of semiconductor storage cells
 - ❖ Processed in words
 - ❖ Address
 - ❖ RAM and memory access time
 - ❖ Memory hierarchy – cache, main memory
 - Secondary storage – larger and cheaper

Arithmetic and Logic Unit (ALU)



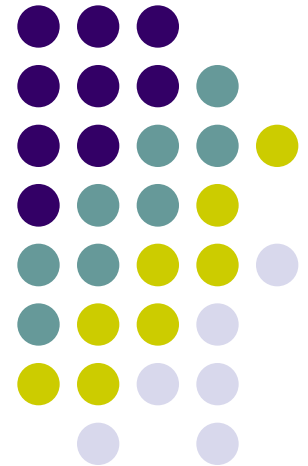
- Most computer operations are executed in ALU of the processor.
- Load the operands into memory – bring them to the processor – perform operation in ALU – store the result back to memory or retain in the processor.
- Registers
- Fast control of ALU

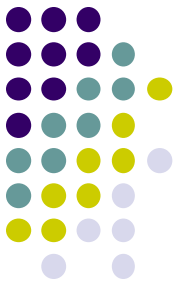
Control Unit



- All computer operations are controlled by the control unit.
- The timing signals that govern the I/O transfers are also generated by the control unit.
- Control unit is usually distributed throughout the machine instead of standing alone.
- Operations of a computer:
 - Accept information in the form of programs and data through an input unit and store it in the memory
 - Fetch the information stored in the memory, under program control, into an ALU, where the information is processed
 - Output the processed information through an output unit
 - Control all activities inside the machine through a control unit

Bus Structures



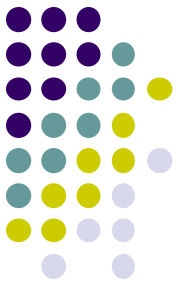


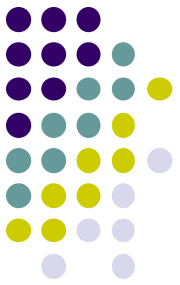
Bus Structures

- There are many ways to connect different parts inside a computer together.
- A group of lines that serves as a connecting path for several devices is called a *bus*.
- Address/data/control

Bus Structure

- Single-bus

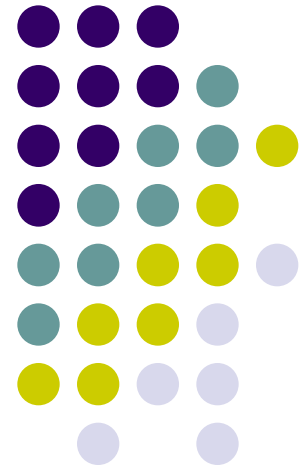


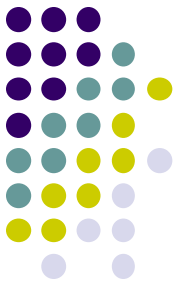


Speed Issue

- Different devices have different transfer/operate speed.
- If the speed of bus is bounded by the slowest device connected to it, the efficiency will be very low.
- How to solve this?
- A common approach – **use buffers.**

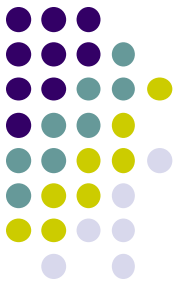
Performance





Performance

- The most important measure of a computer is how quickly it can execute programs.
- Three factors affect performance:
 - Hardware design
 - Instruction set
 - Compiler
- Elapsed Time: $t_5 - t_0$
 - Affected by processor, the disk and printer
 - All units of computer system



Performance

- **Processor time** to execute a program depends on the hardware involved in the execution of individual machine instructions.

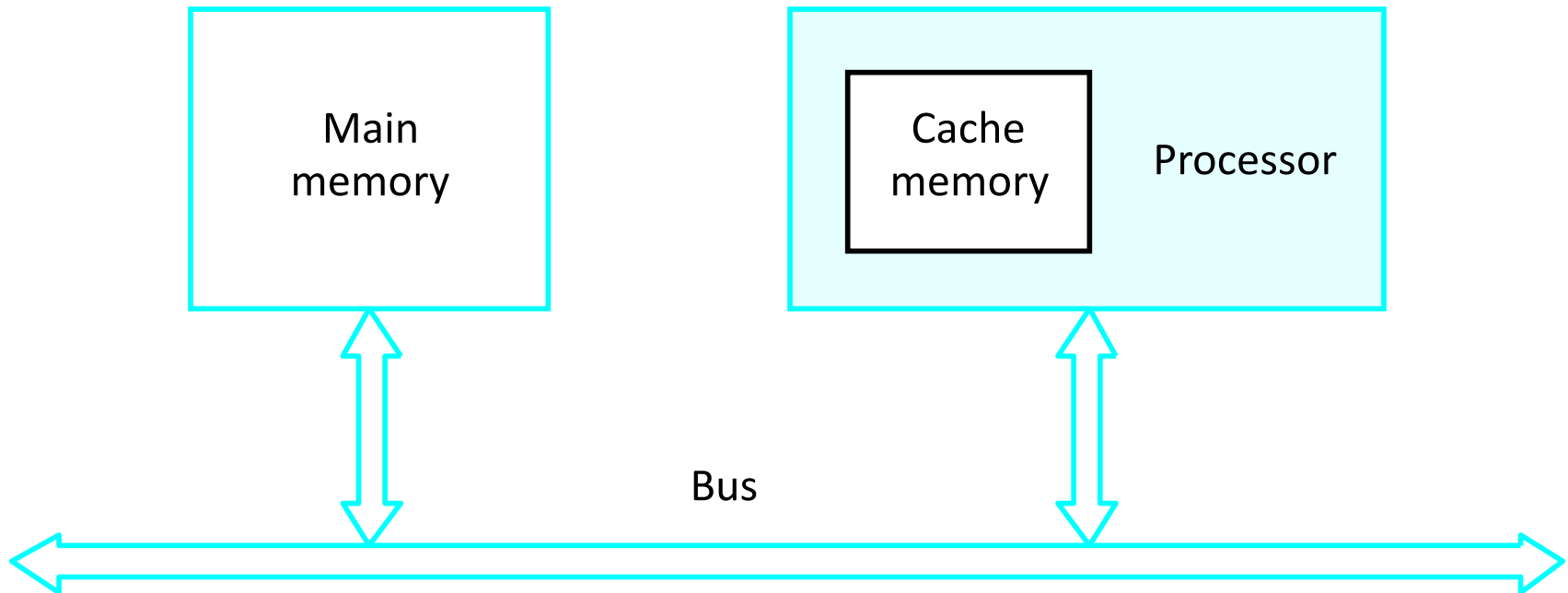
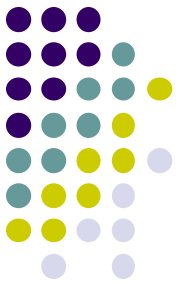


Figure 1.5. The processor cache.

Performance

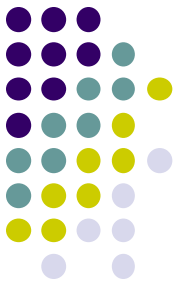


- Program instruction and data are stored in main memory
- Instruction are fetched from main memory over the bus into the cache memory in processor
- Second time read from cache Eg: Program Loop, Data

Processor Clock



- Controlled by timing signals called a **Clock**
- The execution of each instruction is divided into several steps, each of which completes in one clock cycle.
- Regular time intervals called **clock cycle**,
- The **length P** of one clock cycle and
- **clock rate R** = $1/P$, Cycles per second
- Hertz – cycles per second
 - Million instruction /second – Mega
 - Billion instruction /second - Giga



Basic Performance Equation

- T – processor time required to execute a program that has been prepared in high-level language
- N – number of actual machine language instructions needed to complete the execution (note: loop)
- S – average number of basic steps needed to execute one machine instruction. Each step completes in one clock cycle
- R – clock rate
- Note: these are not independent to each other

$$T = \frac{N \times S}{R}$$

How to improve T?



Basic Performance Equation

- Note: these are not independent to each other

$$T = \frac{N \times S}{R}$$

- For better performance- reduce value of N and S and increase R

Pipeline and Superscalar Operation



- Instructions are not necessarily executed one after another.
- The value of S doesn't have to be the number of clock cycles to execute one instruction.
- Pipelining – overlapping the execution of successive instructions.

Eg: Add R1, R2, R3

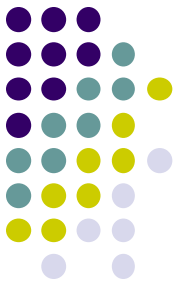
- R1, R2 → input to ALU
- Sum → R3
- Processor can read next instruction
- One instruction/clock cycle

Pipeline and Superscalar Operation

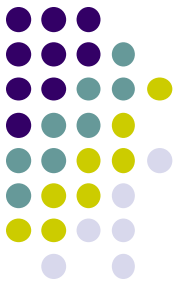


- Effective value of S is 1
- Concurrency
- **Superscalar operation** – multiple instruction pipelines are implemented in the processor in parallel.
- Thus several instructions in one clock cycle called **superscalar Execution**
- Goal – reduce S (could become <1 !)

Clock Rate



- Increase clock rate
 - Improve the integrated-circuit (IC) technology to make the circuits faster
 - Reduce the amount of processing done in one basic step (however, this may increase the number of basic steps needed)
- Increases in R that are entirely caused by improvements in IC technology affect all aspects of the processor's operation equally except the time to access the main memory.
- Cache



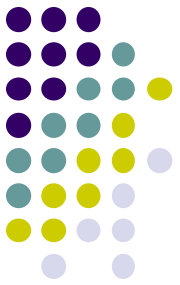
CISC and RISC

- Simple Instruction-small no of steps
- Complex instruction- more no of steps
- Tradeoff between **N and S**
- S must be closed to 1
- Efficient pipelining with simple instruction set
- **RISC-Reduced Instruction set Computers**
- **CISC-Complex Instruction set Computers**



Compiler

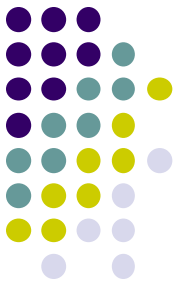
- A compiler translates a high-level language program into a sequence of machine instructions.
- To reduce N , we need a suitable machine instruction set and a compiler that makes good use of it.
- Optimizing compiler Goal – reduce $N \times S$ - **Total no of clock cycles to execute a program**
- A compiler may not be designed for a specific processor; however, a high-quality compiler is usually designed for, and with, a specific processor.



Performance Measurement

- Execution time T is difficult to compute.
- **Computer Community** - Measure computer performance using benchmark programs- standardized programs
- System Performance Evaluation Corporation (**SPEC**) selects and publishes representative application programs for different application domains, together with test results for many commercially available computers.
- Compile and run (no simulation)
- Reference computer Eg: For SPEC95 reference is SUN SPARC station

$$SPEC\ rating = \frac{\text{Running time on the reference computer}}{\text{Running time on the computer under test}}$$

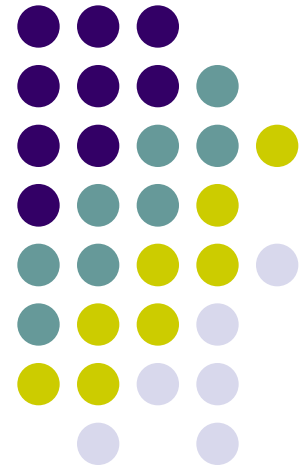


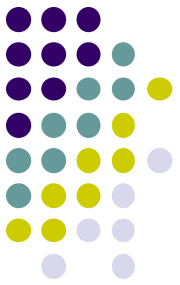
Performance Measurement

- If 50 means , computer is 50 times faster then reference
- Run for all programs in suite (n) and geometric mean is taken as :

$$SPEC\ rating = \left(\prod_{i=1}^n SPEC_i \right)^{\frac{1}{n}}$$

Machine Instructions and Programs

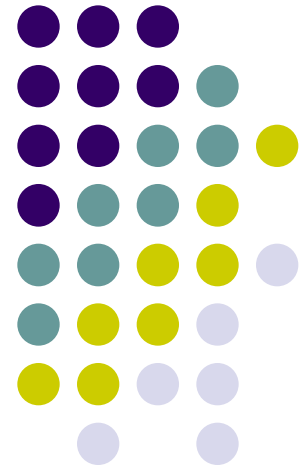


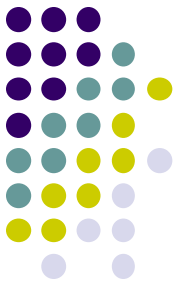


Objectives

- Machine instructions and program execution, including branching and subroutine call and return operations.
- Number representation and addition/subtraction in the 2's-complement system.
Machine Instructions and Programs
- Addressing methods for accessing register and memory operands.
- Assembly language for representing machine instructions, data, and programs.
- Program-controlled Input/Output operations.

Number, Arithmetic Operations, and Characters

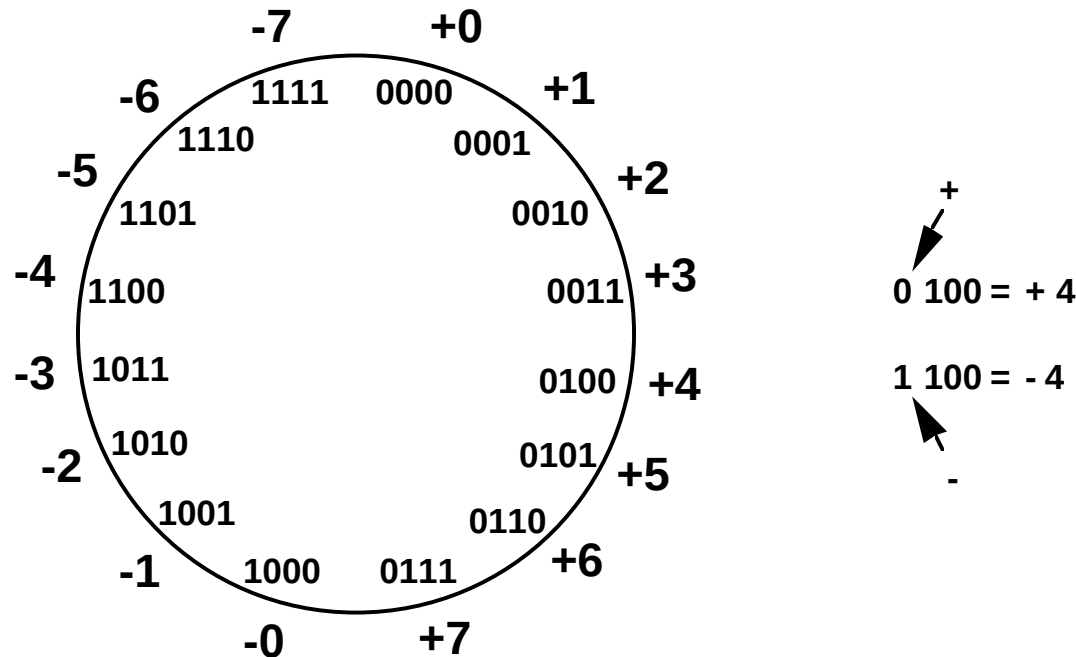
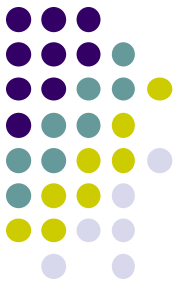




Signed Integer

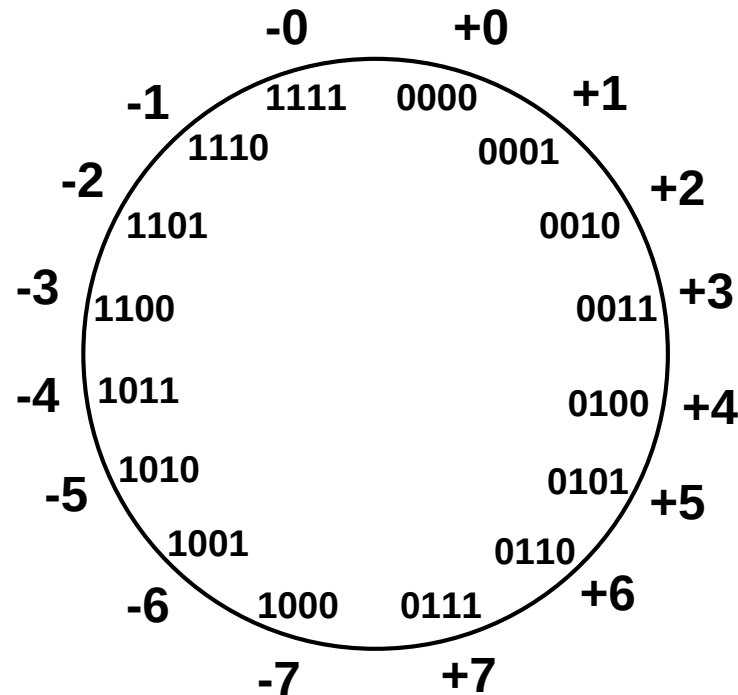
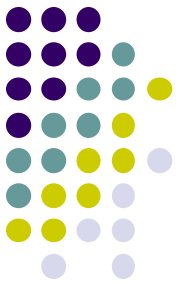
- 3 major representations:
 - Sign and magnitude
 - One's complement
 - Two's complement
- Assumptions:
 - 4-bit machine word
 - 16 different values can be represented
 - Roughly half are positive, half are negative

Sign and Magnitude Representation



High order bit is sign: 0 = positive (or zero), 1 = negative
Three low order bits is the magnitude: 0 (000) thru 7 (111)
Number range for n bits = $\pm 2^{n-1} - 1$
Two representations for 0

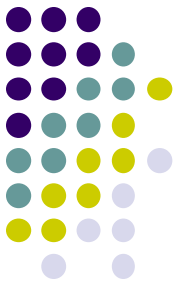
One's Complement Representation



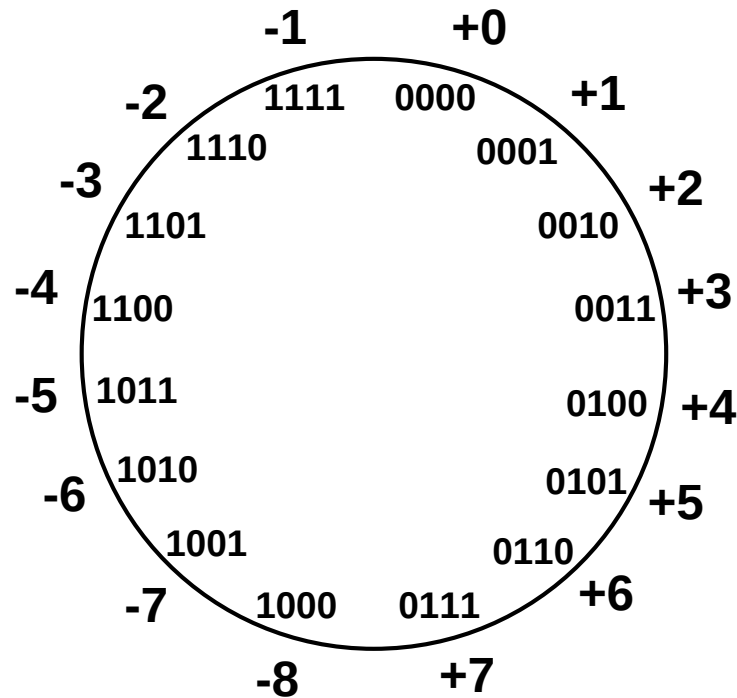
$0\ 100 = +4$
 $1\ 011 = -4$

- Subtraction implemented by addition & 1's complement
- Still two representations of 0! This causes some problems
- Some complexities in addition

Two's Complement Representation



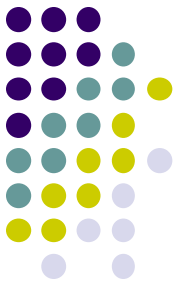
*like 1's comp
except shifted
one position
clockwise*



$0 \ 100 = +4$
 $1 \ 100 = -4$

- Only one representation for 0
- One more negative number than positive number

Binary, Signed-Integer Representations



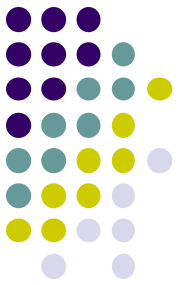
Page 28

<i>B</i>				Values represented		
b_3	b_2	b_1	b_0	Sign and magnitude	1's complement	2's complement
0	1	1	1	+ 7	+ 7	+ 7
0	1	1	0	+ 6	+ 6	+ 6
0	1	0	1	+ 5	+ 5	+ 5
0	1	0	0	+ 4	+ 4	+ 4
0	0	1	1	+ 3	+ 3	+ 3
0	0	1	0	+ 2	+ 2	+ 2
0	0	0	1	+ 1	+ 1	+ 1
0	0	0	0	+ 0	+ 0	+ 0
1	0	0	0	- 0	- 7	- 8
1	0	0	1	- 1	- 6	- 7
1	0	1	0	- 2	- 5	- 6
1	0	1	1	- 3	- 4	- 5
1	1	0	0	- 4	- 3	- 4
1	1	0	1	- 5	- 2	- 3
1	1	1	0	- 6	- 1	- 2
1	1	1	1	- 7	- 0	- 1

Akshata S. Bhayyar, Asst. Prof, CSE Dept. RIT

Figure 2.1. Binary, signed-integer representations.

Addition and Subtraction – 2's Complement



If carry-in to the high order bit = carry-out then ignore carry

$$\begin{array}{r} 4 \quad 0100 \\ + 3 \quad 0011 \\ \hline 7 \quad 0111 \end{array}$$

$$\begin{array}{r} -4 \quad 1100 \\ + (-3) \quad 1101 \\ \hline -7 \quad 11001 \end{array}$$

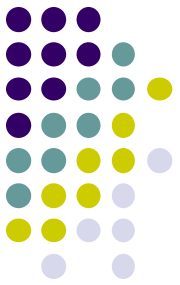
if carry-in differs from carry-out then overflow

$$\begin{array}{r} 4 \quad 0100 \\ - 3 \quad 1101 \\ \hline 1 \quad 10001 \end{array}$$

$$\begin{array}{r} -4 \quad 1100 \\ + 3 \quad 0011 \\ \hline -1 \quad 1111 \end{array}$$

Simpler addition scheme makes twos complement the most common choice for integer number systems within digital systems

2's-Complement Add and Subtract Operations



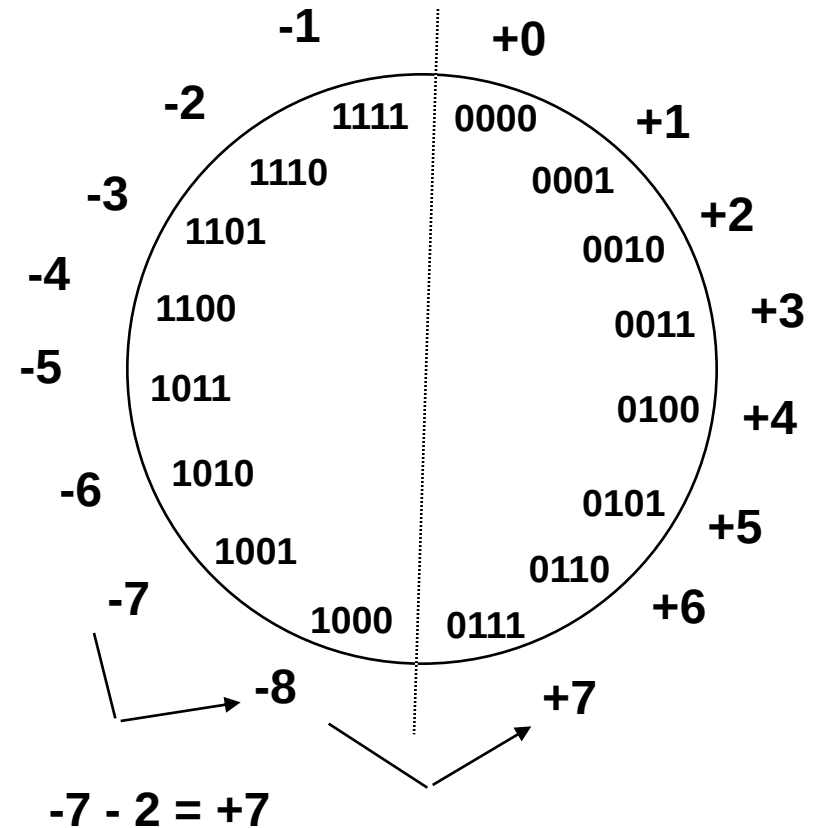
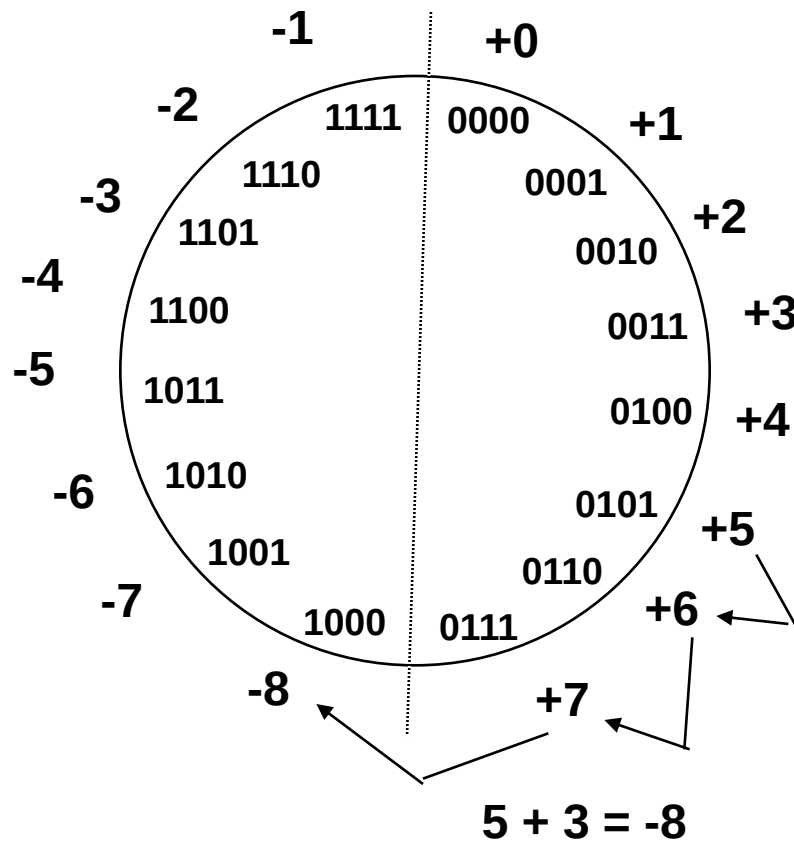
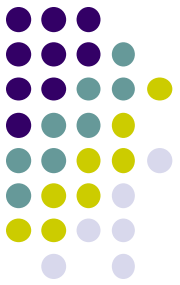
Page 31

(a)	$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array}$	$\begin{array}{r} (+2) \\ (+3) \\ \hline (+5) \end{array}$		(b)	$\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array}$	$\begin{array}{r} (+4) \\ (-6) \\ \hline (-2) \end{array}$
(c)	$\begin{array}{r} 1011 \\ + 1110 \\ \hline 1001 \end{array}$	$\begin{array}{r} (-5) \\ (-2) \\ \hline (-7) \end{array}$		(d)	$\begin{array}{r} 0111 \\ + 1101 \\ \hline 0100 \end{array}$	$\begin{array}{r} (+7) \\ (-3) \\ \hline (+4) \end{array}$
(e)	$\begin{array}{r} 1101 \\ - 1001 \\ \hline \end{array}$	$\begin{array}{r} (-3) \\ (-7) \\ \hline \end{array}$	$\Rightarrow$		$\begin{array}{r} 1101 \\ + 0111 \\ \hline 0100 \end{array}$	$\begin{array}{r} (+4) \\ (+4) \\ \hline (+4) \end{array}$
(f)	$\begin{array}{r} 0010 \\ - 0100 \\ \hline \end{array}$	$\begin{array}{r} (+2) \\ (+4) \\ \hline \end{array}$	$\Rightarrow$		$\begin{array}{r} 0010 \\ + 1100 \\ \hline 1110 \end{array}$	$\begin{array}{r} (+2) \\ (-2) \\ \hline (-2) \end{array}$
(g)	$\begin{array}{r} 0110 \\ - 0011 \\ \hline \end{array}$	$\begin{array}{r} (+6) \\ (+3) \\ \hline \end{array}$	$\Rightarrow$		$\begin{array}{r} 0110 \\ + 1101 \\ \hline 0011 \end{array}$	$\begin{array}{r} (+3) \\ (+3) \\ \hline (+3) \end{array}$
(h)	$\begin{array}{r} 1001 \\ - 1011 \\ \hline \end{array}$	$\begin{array}{r} (-7) \\ (-5) \\ \hline \end{array}$	$\Rightarrow$		$\begin{array}{r} 1001 \\ + 0101 \\ \hline 1110 \end{array}$	$\begin{array}{r} (-2) \\ (-2) \\ \hline (-2) \end{array}$
(i)	$\begin{array}{r} 1001 \\ - 0001 \\ \hline \end{array}$	$\begin{array}{r} (-7) \\ (+1) \\ \hline \end{array}$	$\Rightarrow$		$\begin{array}{r} 1001 \\ + 1111 \\ \hline 1000 \end{array}$	$\begin{array}{r} (-8) \\ (-8) \\ \hline (-8) \end{array}$
(j)	$\begin{array}{r} 0010 \\ - 1101 \\ \hline \end{array}$	$\begin{array}{r} (+2) \\ (-3) \\ \hline \end{array}$	$\Rightarrow$		$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array}$	$\begin{array}{r} (+5) \\ (+5) \\ \hline (+5) \end{array}$

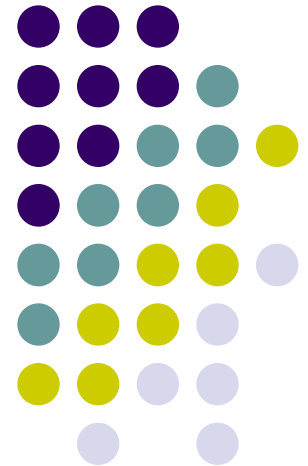
Akshata S. Bhayyar, Asst. Prof, CSE Dept. RIT

Figure 2.4. 2's-complement Add and Subtract operations.

Overflow - Add two positive numbers to get a negative number or two negative numbers to get a positive number



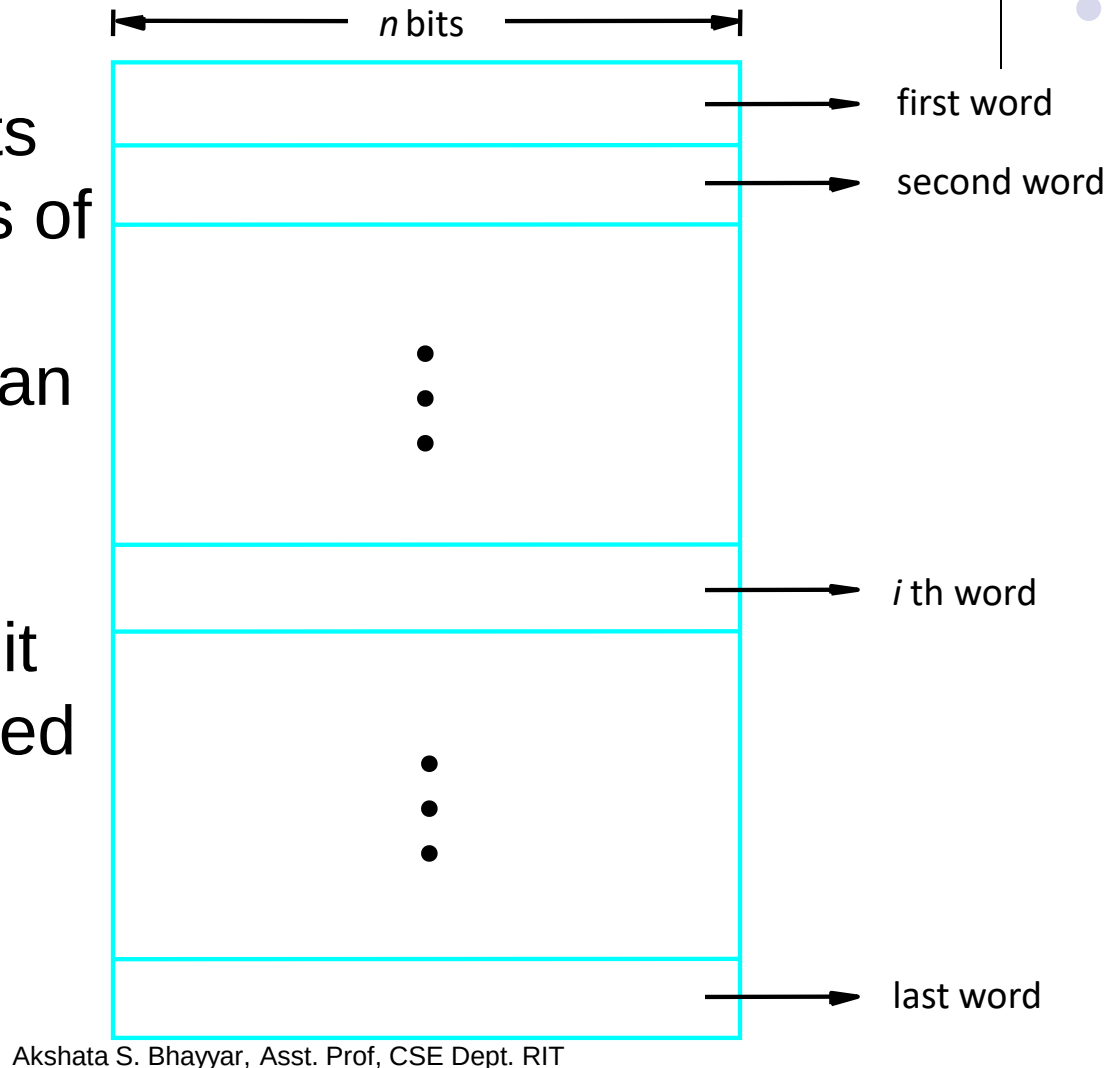
Memory Locations, Addresses, and Operations



Memory Location, Addresses, and Operation



- Memory consists of many millions of storage cells, each of which can store 1 bit.
- Data is usually accessed in n -bit groups. n is called word length.
- 16 bit to 64 bit



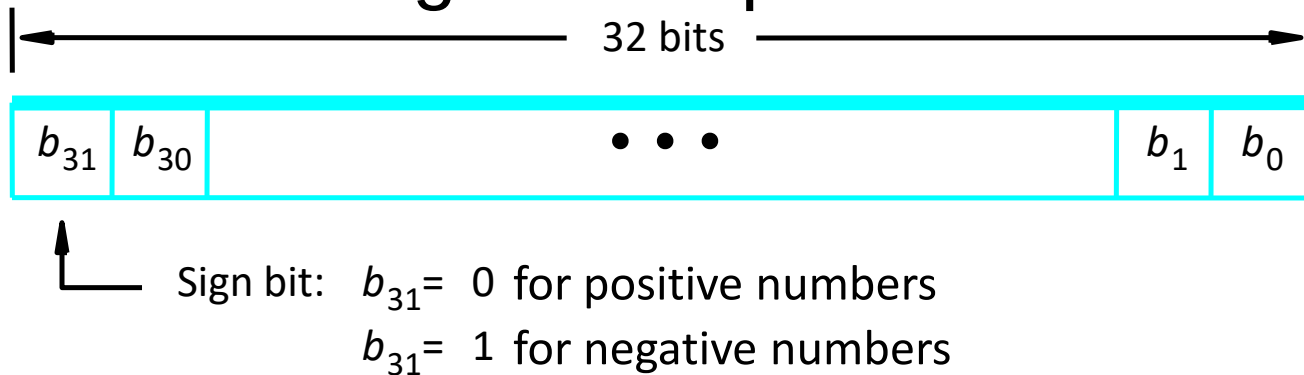
Akshata S. Bhayyar, Asst. Prof, CSE Dept. RIT

Figure 2.5. Memory words.

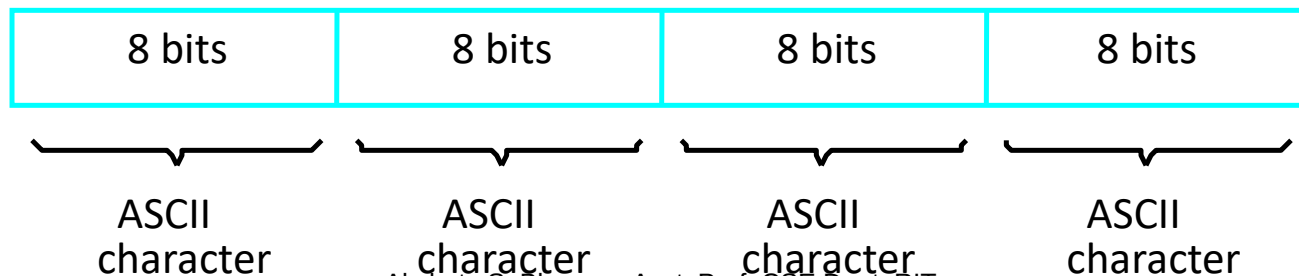
Memory Location, Addresses, and Operation



- 32-bit word length example



(a) A signed integer



(b) Four characters

Memory Location, Addresses, and Operation



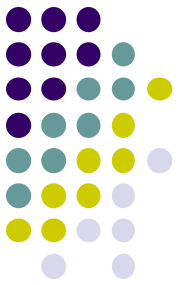
- To retrieve information from memory, either for one word or one byte (8-bit), addresses for each location are needed.
- A k -bit address memory has 2^k memory locations, namely $0 - 2^k - 1$, called memory space.
- 24-bit memory: $2^{24} = 16,777,216$ locations = 16M(16 mega) ($1\text{M}=2^{20}$)
- 32-bit memory: $2^{32} = 4\text{G}$ ($1\text{G}=2^{30}$)
- $1\text{K(kilo)}=2^{10}$
- $1\text{T(tera)}=2^{40}$

Memory Location, Addresses, and Operation



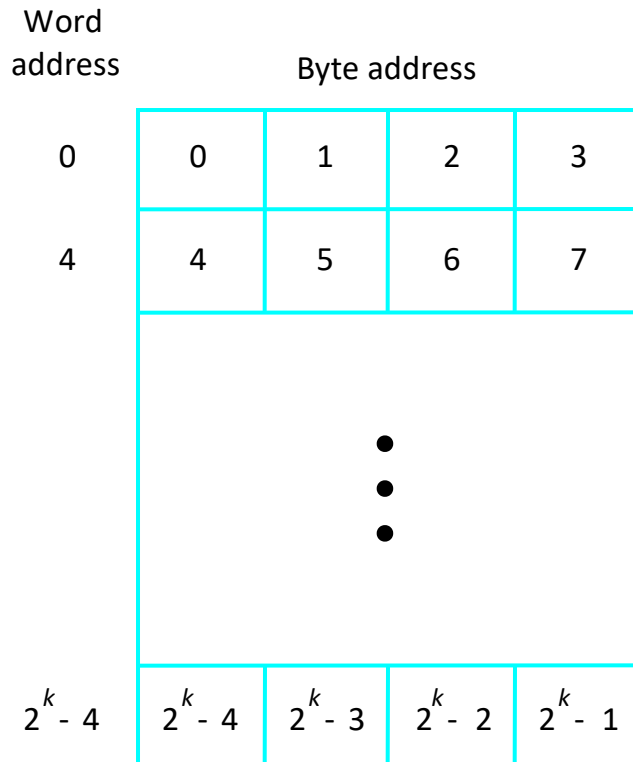
- It is impractical to assign distinct addresses to individual bit locations in the memory.
- The most practical assignment is to have successive addresses refer to successive byte locations in the memory – byte-addressable memory.
- Byte locations have addresses 0, 1, 2, ...
- If word length is 32 bits, then successive words are located at addresses 0, 4, 8, ...

Big-Endian and Little-Endian Assignments

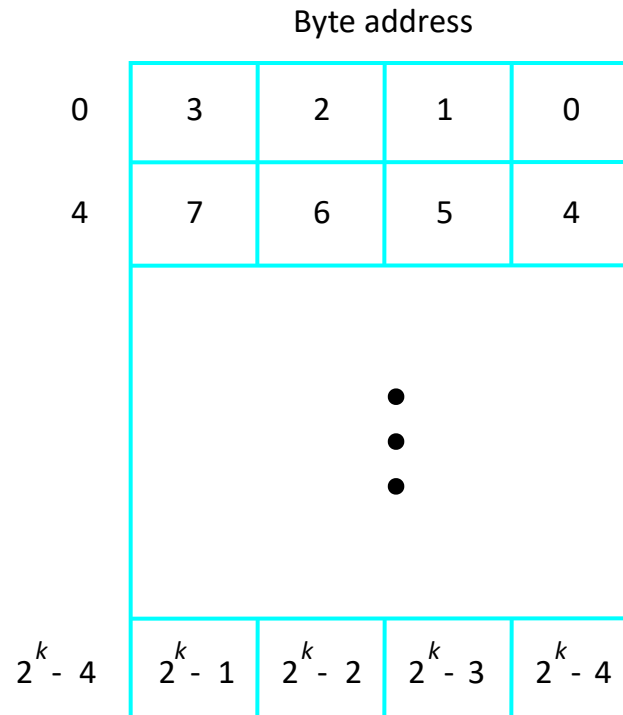


Big-Endian: lower byte addresses are used for the most significant bytes of the word

Little-Endian: opposite ordering. lower byte addresses are used for the less significant bytes of the word



(a) Big-endian assignment



(b) Little-endian assignment

Akshata S. Bhayyar, Asst. Prof, CSE Dept. RIT

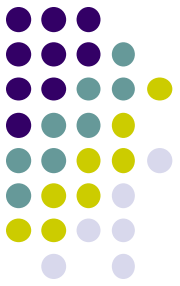
Figure 2.7. Byte and word addressing.

Memory Location, Addresses, and Operation



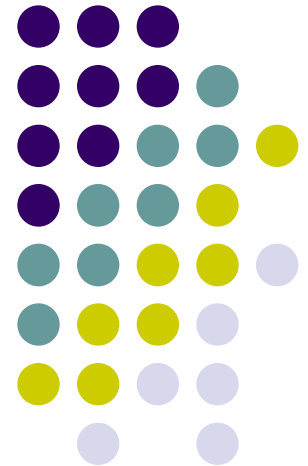
- Address ordering of bytes ($b_7, \dots, b_0$)
- Word alignment
 - Words are said to be aligned in memory if they begin at a byte addr. that is a multiple of the num of bytes in a word.
 - 16-bit word: word addresses: 0, 2, 4,
 - 32-bit word: word addresses: 0, 4, 8,
 - 64-bit word: word addresses: 0, 8, 16,
- Number occupies 1 word-word address
- Access characters and character strings – byte address (C eg –end of string)

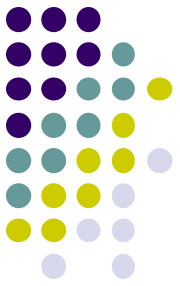
Memory Operation- 2



- **Load (or Read or Fetch)**
 - Copy the content of a specific memory location to the processor.
 - Address – Load
 - Registers can be used
- **Store (or Write)**
 - Transfer information from processor to specific memory location.
 - Overwrite the content in memory
- **Note**
 - One byte /word at a time
 - Processor contains – Registers

Instruction and Instruction Sequencing





“Must-Perform” Operations

- Data transfers between the memory and the processor registers
- Arithmetic and logic operations on data
- Program sequencing and control
- I/O transfers

Register Transfer Notation



- Identify a location by a symbolic name standing for its hardware binary address
(LOC, PLACE, A, VAR2,...)
- Processor Registers - R0,R1,R2,....
- Contents of a location are denoted by placing square brackets around the name of the location

Eg 1. $R1 \leftarrow [LOC]$

Eg 2. $R3 \leftarrow [R1] + [R2]$

- Register Transfer Notation (RTN)

Assembly Language Notation



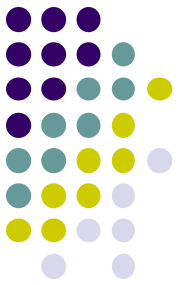
- Represent machine instructions and programs.
- Move LOC, R1 = $R1 \leftarrow [LOC]$
- Add R1, R2, R3 = $R3 \leftarrow [R1] + [R2]$

CPU Organization



- Single Accumulator
 - Result usually goes to the **Accumulator**
 - **Accumulator** has to be saved to memory quite often
- General Register
 - Registers hold operands thus reduce memory traffic
 - Register bookkeeping
- Stack
 - Operands and result are always in the stack

Instruction Formats



- Three-Address Instructions

- ADD R1, R2, R3 $R1 \leftarrow R2 + R3$

Operation Source1,Source2,Destination

- Two-Address Instructions

- ADD R1, R2 $R1 \leftarrow R1 + R2$

Operation Source,Destination

- Move R2,R3
Add R1,R3

Instruction Formats



- One-Address Instructions

- ADD B

$AC \leftarrow AC + M[B]$

- $C \leftarrow [A] + [B]$

Load A

Add B

Store C

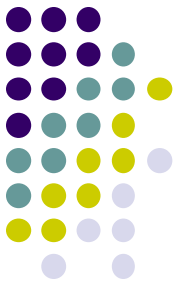
Instruction Formats

- Zero-Address Instructions

- ADD

$\text{TOS} \leftarrow \text{TOS} + (\text{TOS} - 1)$



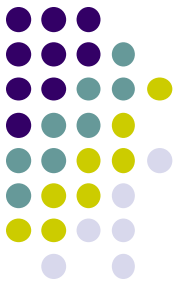


Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Three-Address

- | | | | |
|----|-----|-----------|-------------------------------|
| 1. | ADD | R1, A, B | ; $R1 \leftarrow M[A] + M[B]$ |
| 2. | ADD | R2, C, D | ; $R2 \leftarrow M[C] + M[D]$ |
| 3. | MUL | X, R1, R2 | ; $M[X] \leftarrow R1 * R2$ |

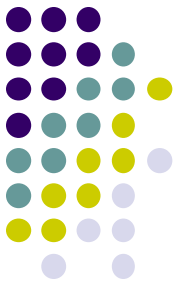


Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Two-Address

- | | | | |
|----|-----|--------|-----------------------------|
| 1. | MOV | R1, A | ; $R1 \leftarrow M[A]$ |
| 2. | ADD | R1, B | ; $R1 \leftarrow R1 + M[B]$ |
| 3. | MOV | R2, C | ; $R2 \leftarrow M[C]$ |
| 4. | ADD | R2, D | ; $R2 \leftarrow R2 + M[D]$ |
| 5. | MUL | R1, R2 | ; $R1 \leftarrow R1 * R2$ |
| 6. | MOV | X, R1 | ; $M[X] \leftarrow R1$ |

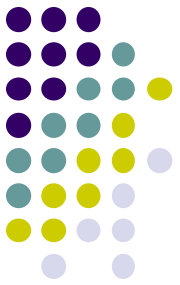


Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- One-Address

- | | |
|-----------|-----------------------------|
| 1. LOAD A | ; $AC \leftarrow M[A]$ |
| 2. ADD B | ; $AC \leftarrow AC + M[B]$ |
| 3. STORET | ; $M[T] \leftarrow AC$ |
| 4. LOAD C | ; $AC \leftarrow M[C]$ |
| 5. ADD D | ; $AC \leftarrow AC + M[D]$ |
| 6. MUL T | ; $AC \leftarrow AC * M[T]$ |
| 7. STOREX | ; $M[X] \leftarrow AC$ |

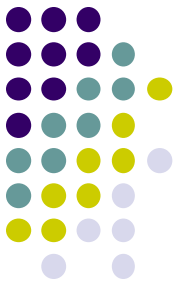


Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Zero-Address

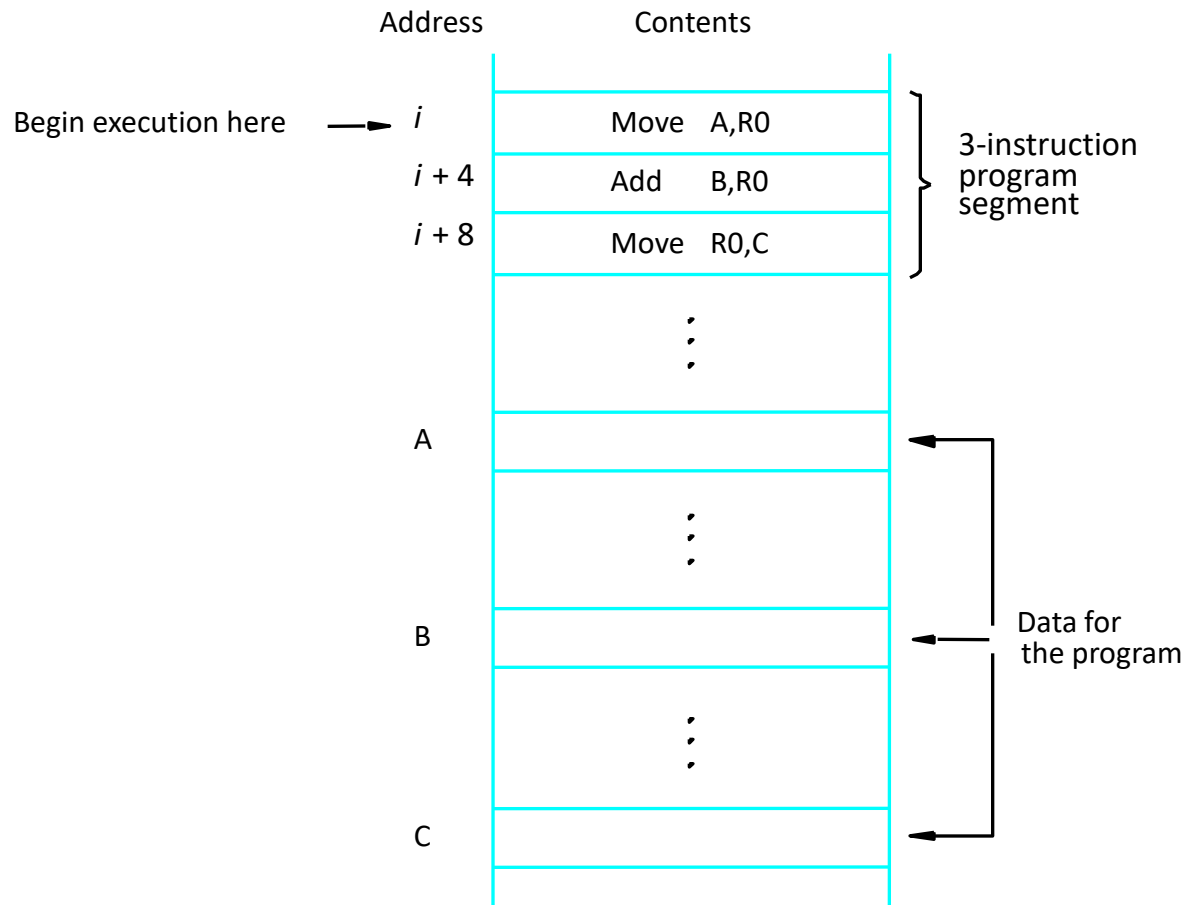
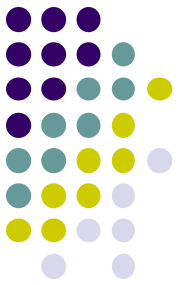
- | | |
|-------------|----------------------------|
| 1. PUSH A | ; TOS $\leftarrow$ A |
| 2. PUSH B | ; TOS $\leftarrow$ B |
| 3. ADD | ; TOS $\leftarrow$ (A + B) |
| 4. PUSH C | ; TOS $\leftarrow$ C |
| 5. PUSH D | ; TOS $\leftarrow$ D |
| 6. ADD | ; TOS $\leftarrow$ (C + D) |
| 7. MUL | ; TOS $\leftarrow$ |
| (C+D)*(A+B) | |
| 8. POP X | ; M[X] $\leftarrow$ TOS |



Using Registers

- Registers are faster
- Shorter instructions
 - The number of registers is smaller (e.g. 32 registers need 5 bits)
- Potential speedup
- Minimize the frequency with which data is moved back and forth between the memory and processor registers.

Instruction Execution and Straight-Line Sequencing



Assumptions:

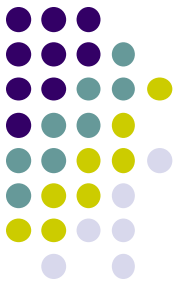
- One memory operand per instruction
- 32-bit word length
- Memory is byte addressable
- Full memory address can be directly specified in a single-word instruction

Two-phase procedure

- Instruction fetch
- Instruction execute

Page 43

Instruction Execution and Straight-Line Sequencing



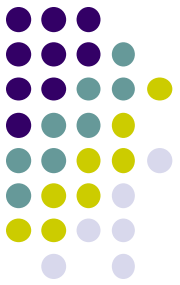
Two-phase procedure

-Instruction fetch : Fetched from memory location whose address is in PC and then placed in IR

-Instruction execute: Execute instruction from IR

- Fetching operands
- Perform operation
- Storing result

Branching



i	Move	NUM1,R0
$i + 4$	Add	NUM2,R0
$i + 8$	Add	NUM3,R0
	⋮	
$i + 4n - 4$	Add	NUM n ,R0
$i + 4n$	Move	R0,SUM
	⋮	
SUM		
NUM1		
NUM2		
	⋮	
NUM n		

Akshata S. Bhayyar, Asst. Prof, CSE Dept. RIT

Figure 2.9. A straight-line program for adding n numbers.

Branching

Branch target

Conditional branch

Program loop

LOOP

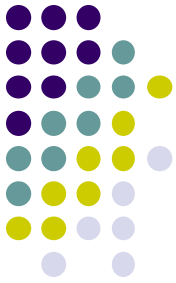
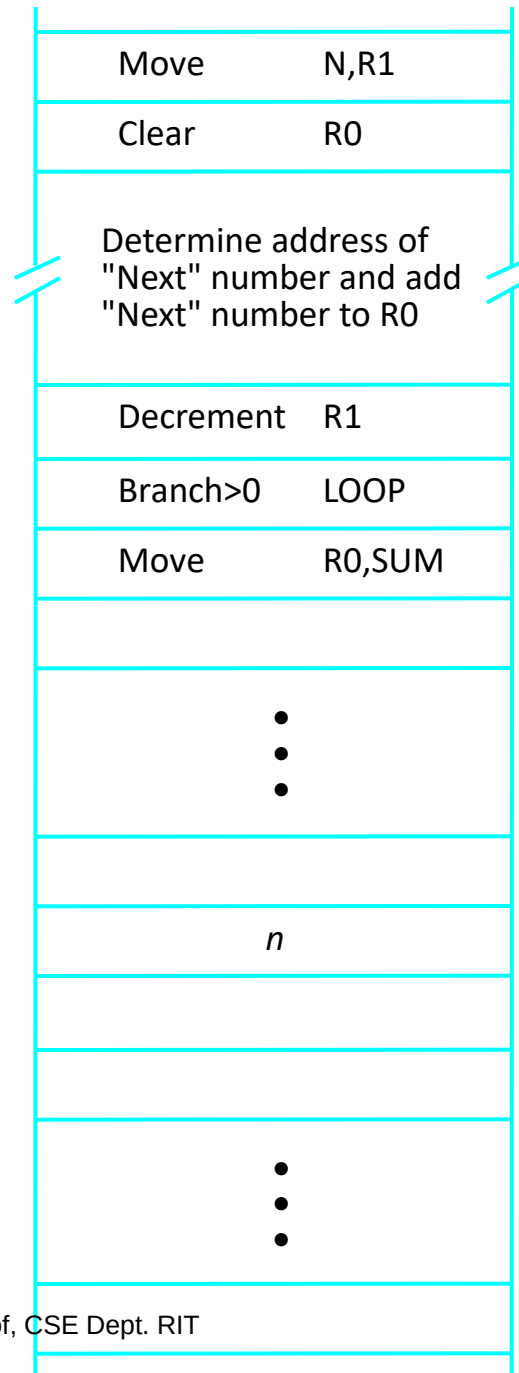
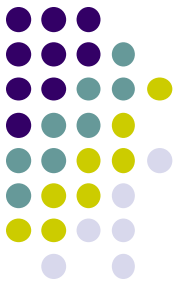


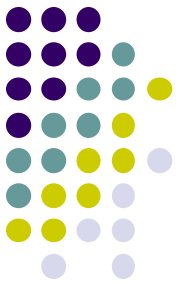
Figure 2.10. Using a loop to add n numbers.



Condition Codes

- Condition code flags
- Condition code register / status register
- Four commonly used : set to 1 or 0
 - N (negative)
 - Z (zero)
 - V (overflow)
 - C (carry)
- Different instructions affect different flags

Conditional Branch Instructions



- Example: A-B

- A: 1 1 1 1 0 0 0 0

- B: 0 0 0 1 0 1 0 0

A: 1 1 1 1 0 0 0 0

+(-B): 1 1 1 0 1 1 0 0

1 1 0 1 1 1 0 0

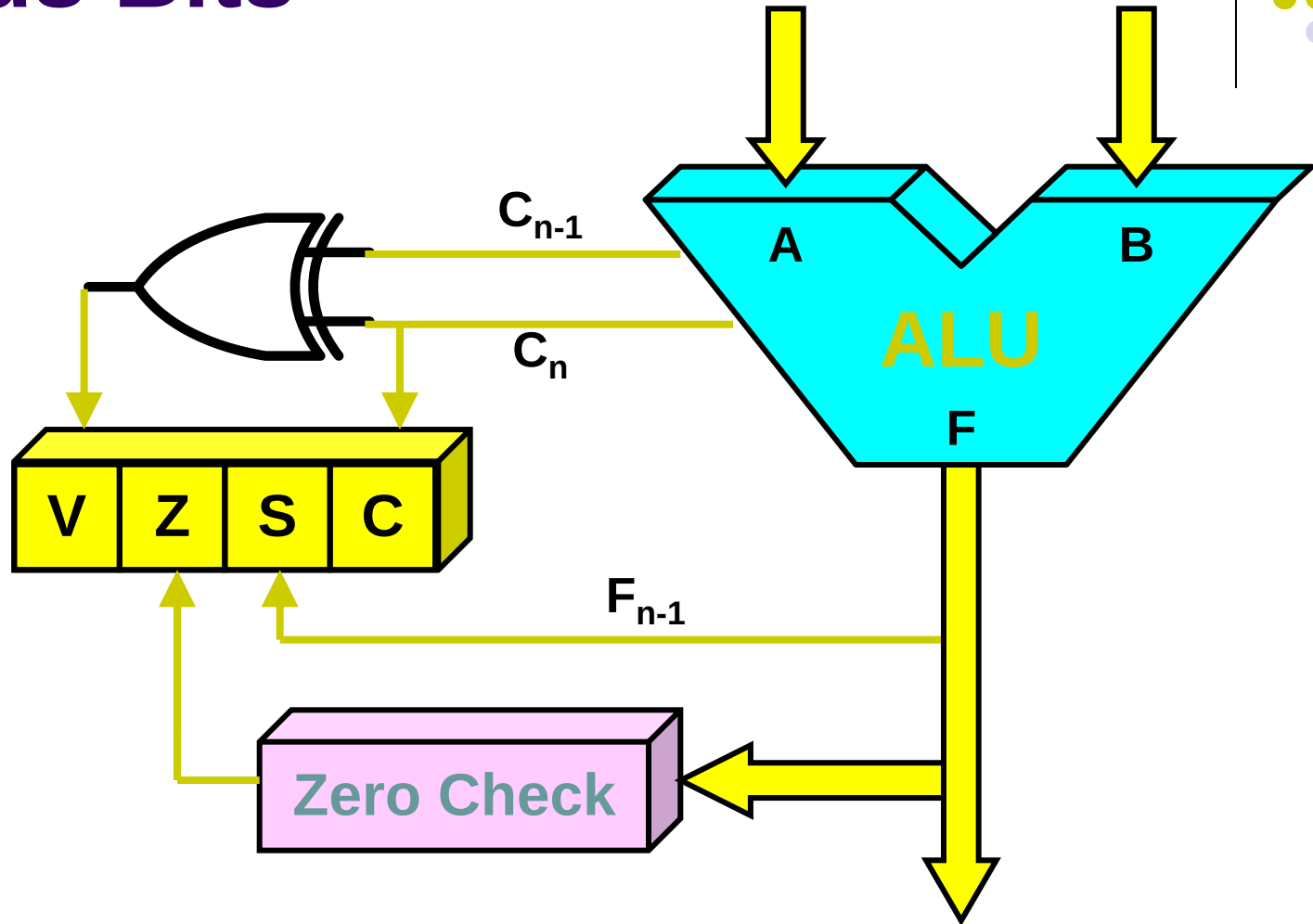
C = 1

Z = 0

S = 1

V = 0

Status Bits





Addressing Modes

Addressing Modes



- The way in which location of the operand is specified in an instruction.

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	Ri	EA = Ri
Absolute (Direct)	LOC	EA = LOC
Indirect	(Ri)	EA = [Ri]
	(LOC)	EA = [LOC]
Index	X(Ri)	EA = [Ri] + X
Base with index	(Ri,Rj)	EA = [Ri] + [Rj]
Base with index and offset	X(Ri,Rj)	EA = [Ri] + [Rj] + X
Relative	X(PC)	EA = [PC] + X
Autoincrement	(Ri)+	EA = [Ri]; Increment Ri
Autodecrement	-(Ri)	Decrement Ri; EA = [Ri]

EA = effective address

Value = a signed number

Fig. 1.7 Addressing modes

Addressing Modes



1. Immediate mode

- The use of a constant in

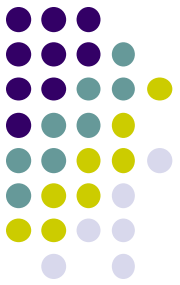
Move #200, R1, i.e. $R1 \leftarrow 200$

Move B,R1

Add #6,R1

Move R1,A

Addressing Modes



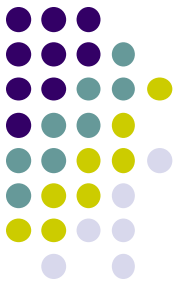
- Variable
- Accessing variable
 2. **Register mode:** By name of register Eg:
Move R1, R2
 3. **Absolute mode:** By address of memory location Eg: Move LOC, R1

Addressing Modes



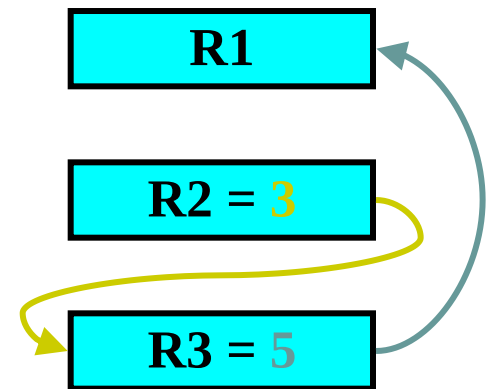
4. Indirection and Pointers. mode

- Provide information from where the memory address of the operand can be determined-
Effective Address

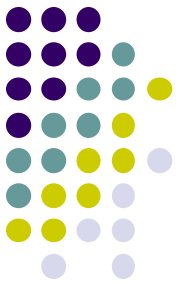


Addressing Modes

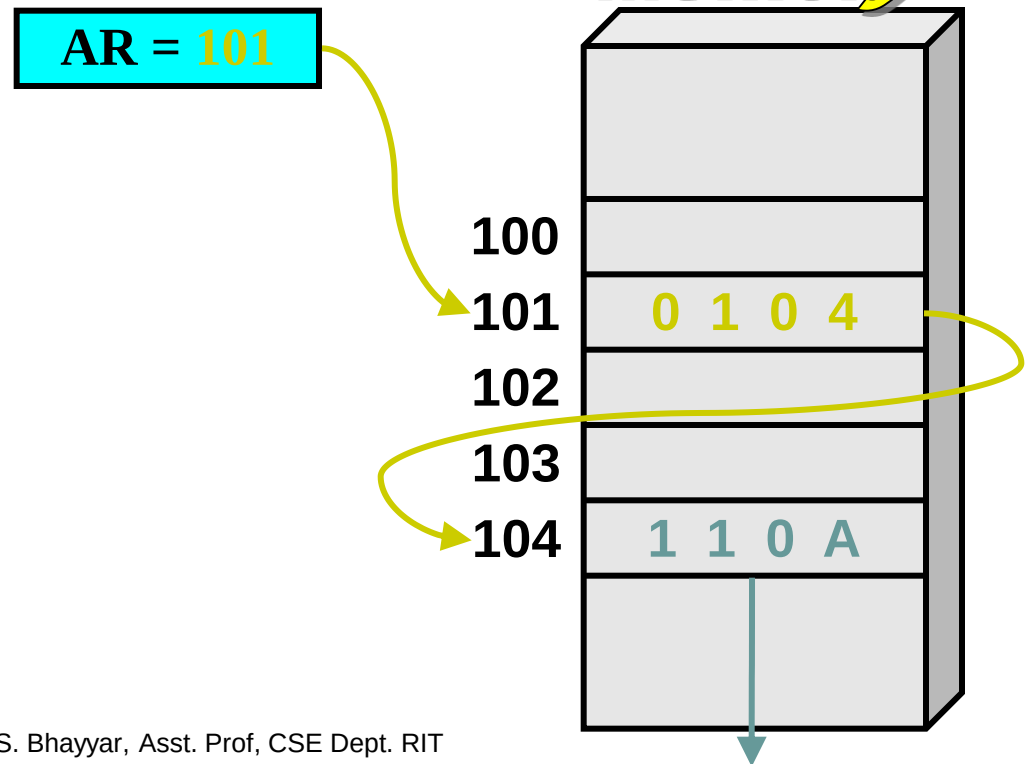
- Register Indirect
 - Indicate the register that holds the address of the register that holds the operand
MOV (R2), R1
- The register /memory location that contain the address— **pointer**
Eg: Treasure Hunt



Addressing Modes



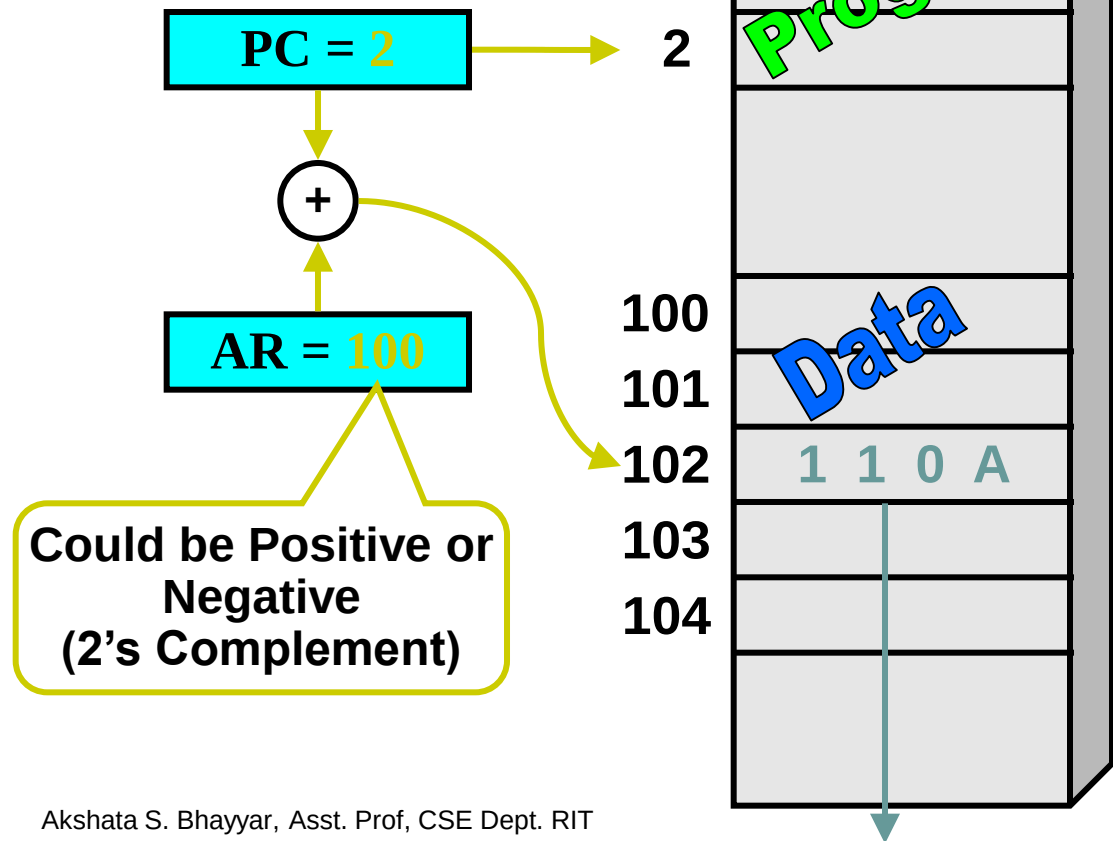
- Indirect Address
 - Indicate the memory location that holds the address of the memory location that holds the data

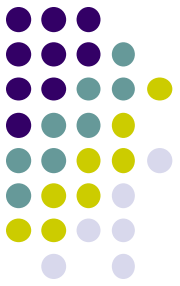


Addressing Modes

5. Relative Address

- $EA = PC + \text{Relative Addr}$

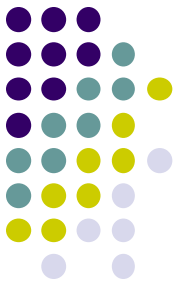




Addressing Modes

6 Direct Address

- Use the given address to access a memory location
- Move R1,R2



7. Indexing and Arrays

- **Index mode** – the effective address of the operand is generated by adding a constant value to the contents of a register.
- **Index register**- general purpose register
- $X(R_i)$: $EA = X + [R_i]$
- The constant X may be given either as an explicit number or as a symbolic name representing a numerical value.
- X is called **displacement**

Indexing and Arrays

Engineered for Tomorrow

1. Offset is given as a constant

Add 20(R1),R2

1. Offset is in the index register

Add 1000(R1), R2

Indexed addressing

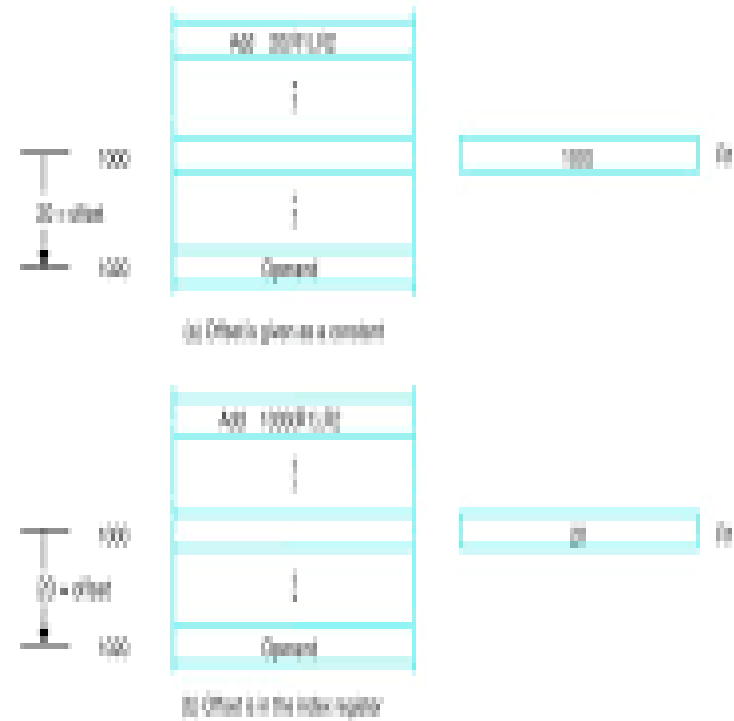
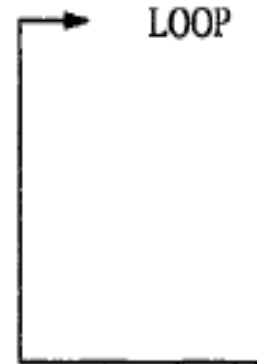
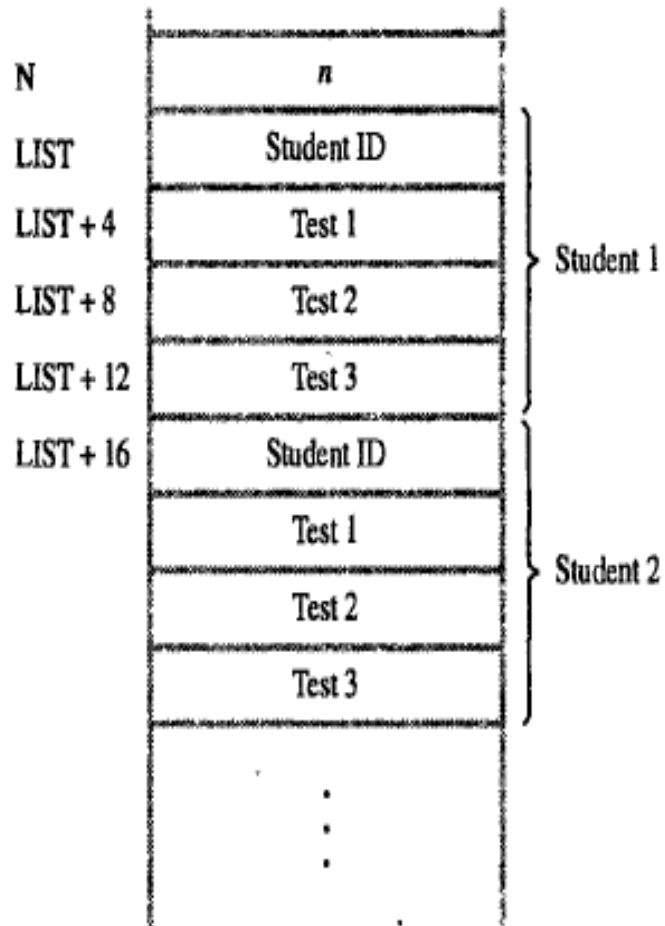
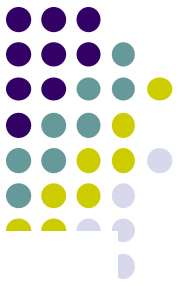


Figure 2.13. Indexed addressing

Indexing and Arrays



```

Move    #LIST,R0
Clear   R1
Clear   R2
Clear   R3
Move    N,R4
Add     4(R0),R1
Add     8(R0),R2
Add     12(R0),R3
Add     #16,R0
Decrement R4
Branch>0 LOOP
Move    R1,SUM1
Move    R2,SUM2
Move    R3,SUM3
  
```

7. Indexing and Arrays



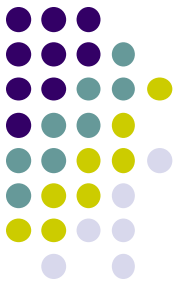
- In general, the Index mode facilitates access to an operand whose location is defined relative to a reference point within the data structure in which the operand appears.

- **Several variations:**

1. $(R_i, R_j): EA = [R_i] + [R_j]$

second register contain offset value called as Base register

2. $X(R_i, R_j): EA = X + [R_i] + [R_j]$



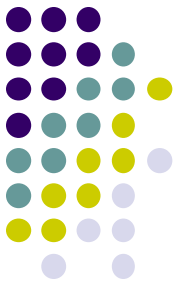
8. Relative Addressing

- Relative mode – the effective address is determined by the Index mode using the **program counter** in place of the general-purpose register.

- $X(PC)$ – note that X is a signed number

Eg Branch>0 LOOP

- This location is computed by specifying it as an offset from the current value of PC.
- Branch target may be either before or after the branch instruction.



9. Additional Modes

- **Autoincrement mode** – the effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list.
- $(R_i)+$. The increment is 1 for byte-sized operands, 2 for 16-bit operands, and 4 for 32-bit operands.
- **Autodecrement mode**: $-(R_i)$ – decrement first

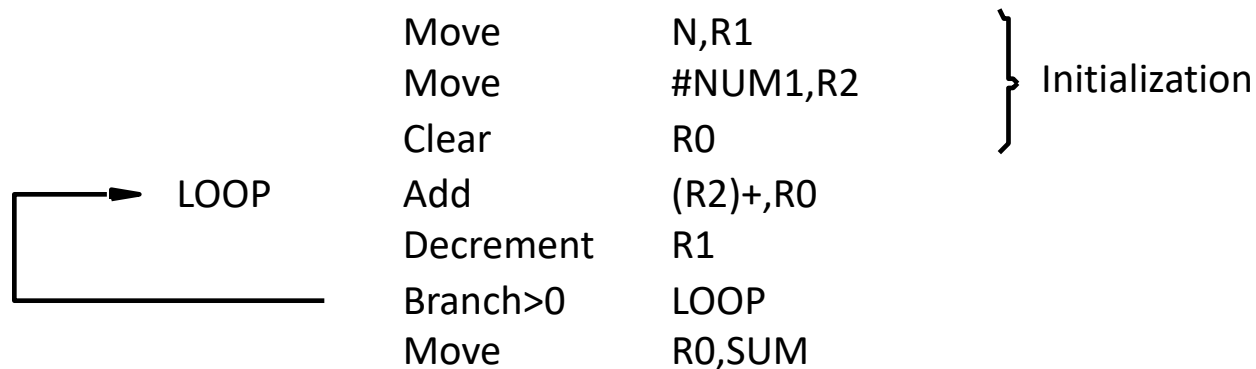
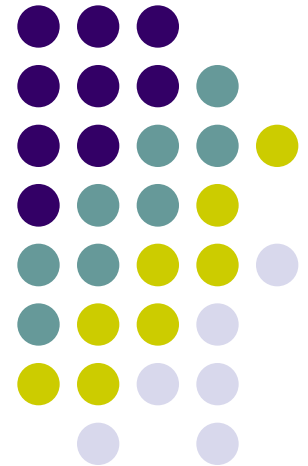


Figure 2.16. The Autoincrement addressing mode used in the program of Figure 2.12.

Encoding of Machine Instructions



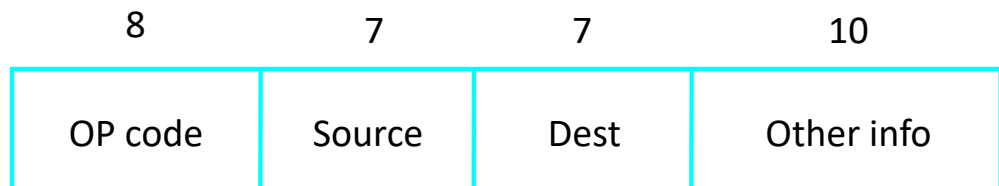
Encoding of Machine Instructions



- Instruction must be encoded – **binary pattern** --- called as **machine instruction**
- An assumption was made that all instructions are **one word in length**.
- **OP code**: the type of operation to be performed and the type of operands used may be specified using an encoded binary pattern
- Suppose 32-bit word length,
 - 8-bit OP code (how many instructions can we have?),
 - 16 registers in total (how many bits?),
 - 3-bit addressing mode indicator.

Example 1

- Add R1, R2



(a) One-word instruction

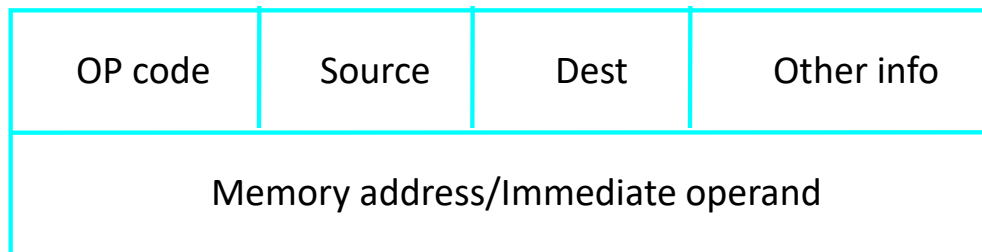
Encoding of Machine Instructions



- What happens if we want to specify a memory operand using the Absolute addressing mode?

Eg :Move R2, LOC

- 18 bit for OPCode and register, 14-bit for LOC – insufficient
- **Solution** – use two words



(b) Two-word instruction

Encoding of Machine Instructions

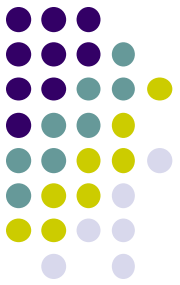


- Then what if an instruction in which two operands can be specified using the Absolute addressing mode?

Eg : Move LOC1, LOC2

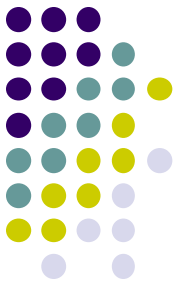
- **Solution** – use two additional words
- This approach results in instructions of variable length. Complex instructions can be implemented, closely resembling operations in high-level programming languages – **Complex Instruction Set Computer (CISC)**

Encoding of Machine Instructions



- If we insist that all instructions must fit into a single 32-bit word, it is not possible to provide a 32-bit address or a 32-bit immediate operand within the instruction.
- It is still possible to define a highly functional instruction set, which makes extensive use of the processor registers.
- Add R1, R2 ----- yes
- Add LOC, R2 ----- no
- Add (R3), R2 ----- yes--- Problem How to store address in R3 register

Encoding of Machine Instructions



- **Solution**

1. Store the address of operand memory location close to the program.
2. Use logical and shift instructions

Encoding of Machine Instructions



- The restriction to use one word has led to Reduced Instruction Set Computers(RISC)

Thus previous instruction can be written as :

Move (R3), R1

Add R1, R2

We can also use

Add R1, R2, R3

OPcode	R1	R2	Rk	Other info
--------	----	----	----	---------------